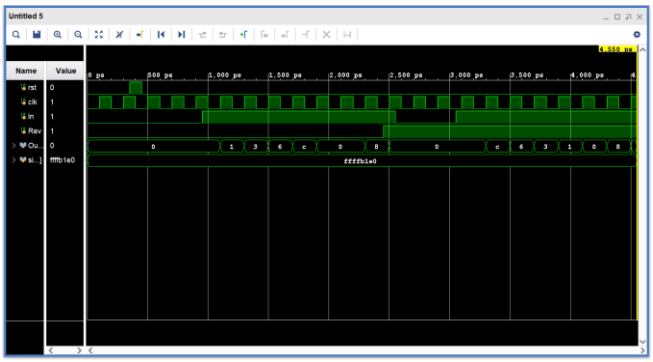
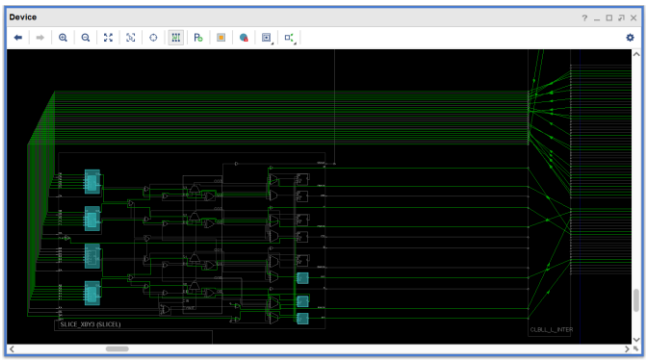
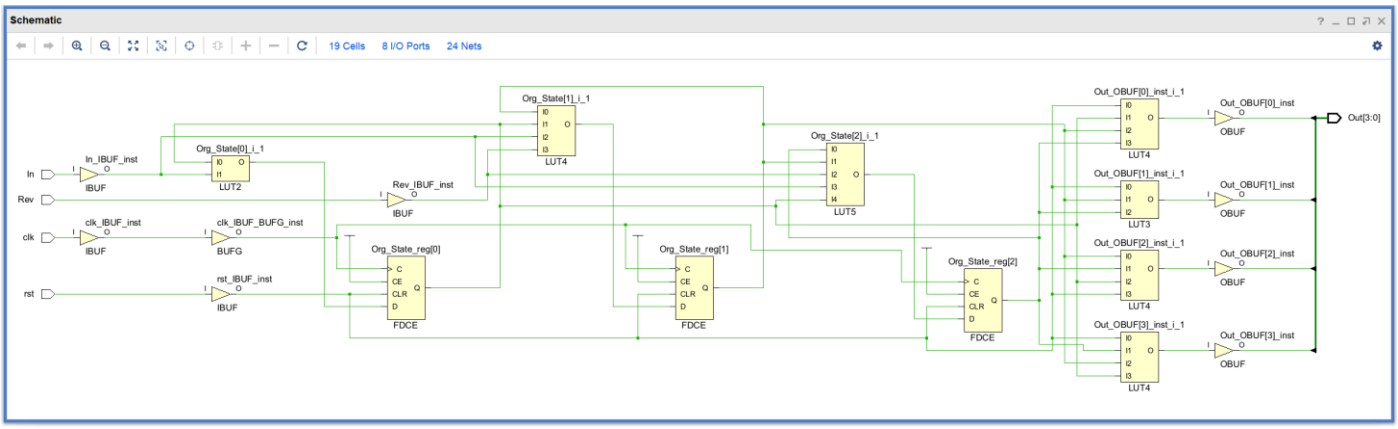
第三次作業(2026/04/09)

學號:0136C008　　姓名:涂名恩

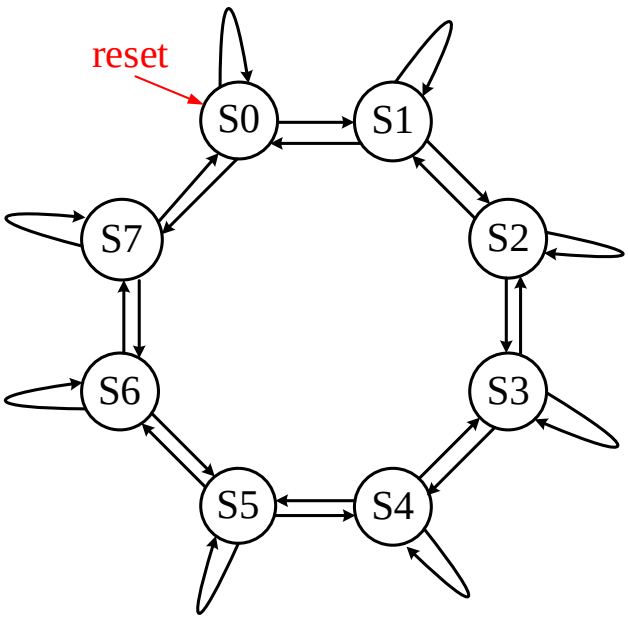
1. 使用有限狀態機(FSM)的方法完成顯示學號的電路，需要具備正數，倒數，暫停的 功能。







2. 修改顯示學號的電路，使它能顯示累加的結果。如果原本在 S0, S1, ⋯, S7 各個狀態依序顯示 A0, A1, ⋯, A7，修改後的電路會顯示 A0, A0+A1, A0+A1+A2, ⋯, $\sum_{k=0}^{7} A_k$.



圖一. 狀態機轉換圖

(a) 請在報告詳細說明電路的設計方法，包含狀態的設定，輸出與狀態和輸入的關係，以及如何控制狀態的轉換。可以參考圖一畫出狀態轉換圖。(15%*2)

設計方法：

有限狀態機依序顯示學號各位數值的累加結果。

狀態的設定：

狀態	對應數值	累加結果
S0	A0 = 0	0
S1	A1 = 1	1
S2	A2 = 3	4
S3	A3 = 6	10
S4	A4 = C(12)	22
S5	A5 = 0	22
S6	A6 = 0	22
S7	A7 = 8	30

輸出與狀態和輸入的關係：

輸出僅與當前狀態有關，與輸入無關。

如何控制狀態的轉換：

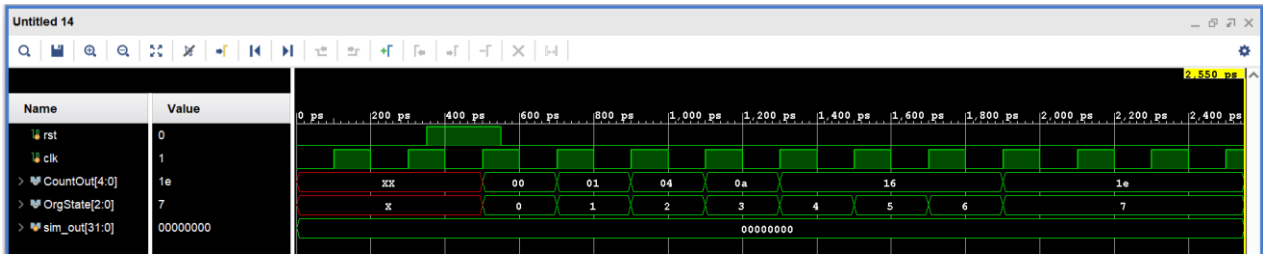
狀態轉換由時脈 clk 控制，並在每個 clock 上升沿更新。

當 rst = 1 時，系統會初始化 S0。

(b) 請設計對應的測試環境(test bench, 例如 TB_FSM.v)，並說明如何驗證電路的功能，並進行模擬以確認電路功能正確。(15%*2)

如何驗證電路的功能：

1. FSM 回到初始狀態 S0
2. reset 解除後，FSM 應依序由 S0 跳到 S7
3. 確認輸出是否符合累加結果(00 → 01 → 04 → 0A → 16 → 16 → 16 → 1E)

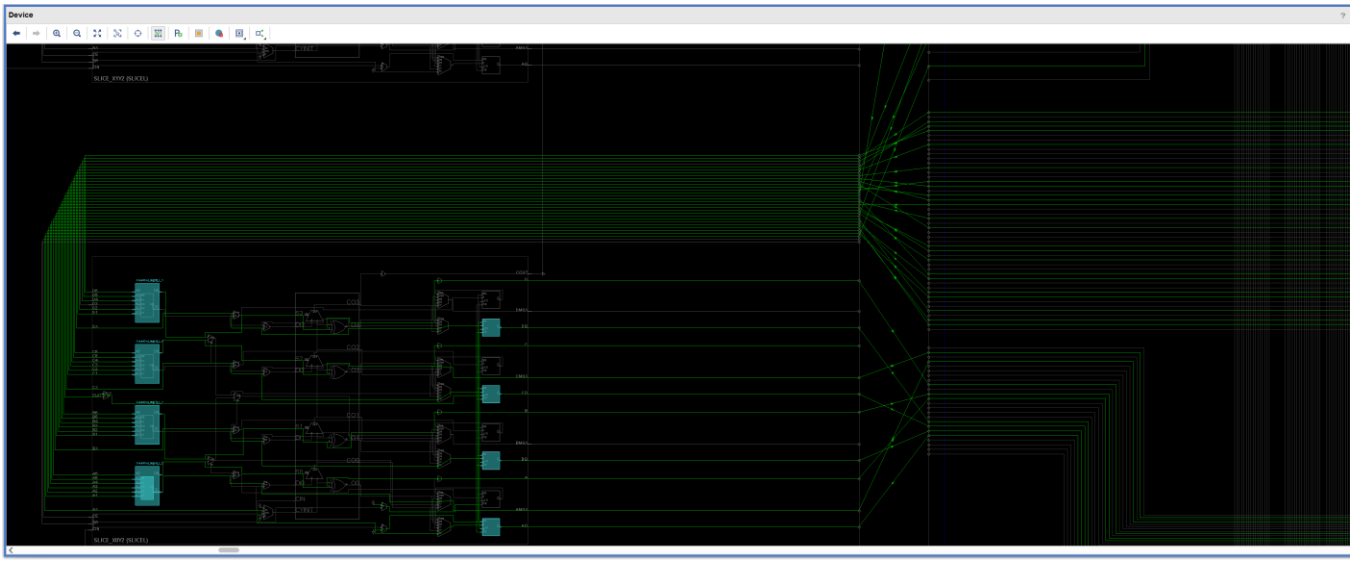


```verilog
`timescale 1ns / 1ps
module FSM_HW_2(clk, rst, CountOut, Org_State);
input clk, rst;
output [4:0] CountOut;
reg [4:0] CountOut;
output [2:0] Org_State;

// State machine states
parameter S0 = 3'b000;
parameter S1 = 3'b001;
parameter S2 = 3'b010;
parameter S3 = 3'b011;
parameter S4 = 3'b100;
parameter S5 = 3'b101;
parameter S6 = 3'b110;
parameter S7 = 3'b111;

reg [2:0] Org_State;
reg [2:0] New_State;

//==== Org_State ===
//always @(posedge rst or posedge clk) begin
always @(posedge clk) begin
    if( (rst == 1'b1)) begin
        Org_State <= S0;
    end
    else begin
        Org_State <= New_State;
    end
end

//==== New_State ===
always @(Org_State) begin
    case ( Org_State )
        S0: New_State = S1;
        S1: New_State = S2;
        S2: New_State = S3;
        S3: New_State = S4;
        S4: New_State = S5;
        S5: New_State = S6;
        S6: New_State = S7;
        S7: New_State = S7;
        default: New_State = S0;
    endcase
end

//==== Out ===
always @(Org_State) begin
    case ( Org_State )
        S0: CountOut = 5'h0;
        S1: CountOut = CountOut + 5'h1;
        S2: CountOut = CountOut + 5'h3;
        S3: CountOut = CountOut + 5'h6;
        S4: CountOut = CountOut + 5'hC;
        S5: CountOut = CountOut + 5'h0;
        S6: CountOut = CountOut + 5'h0;
        S7: CountOut = CountOut + 5'h8;
        default: CountOut = 5'b00000;
    endcase
end

endmodule
```

```verilog
`timescale 10ps / 1ps
module TB_FSM_HW_2;

reg rst, clk;
wire [4:0] CountOut;

wire [2:0] OrgState;

    integer sim_out;

    FSM_HW_2 U_counter(.clk(clk), .rst(rst), .CountOut(CountOut), .Org_State(OrgState));

    always begin
#10
clk=~clk;
end

initial begin
    sim_out=$fopen("C:/Users/Administrator/Desktop/Classes/NTOU_20250201/Verilog_202502/Week7/Test_1/SimOut2.txt", "w");
    rst= 1'b0;
    clk= 1'b0;

    #35  rst=1'b1;
    #20  rst=1'b0;
    #200 $fclose(sim_out);
        $stop;
end


always @(posedge clk) begin
    //$fwrite(sim_out, "rst=%b, CurrentState= 3'o%o, NextState= 3'o%o, CountOut= 4'h%h\n",
    $fwrite(sim_out, "rst=%b, CountOut= 5'h%h\n",
        rst,
        //U_counter.Org_State[2:0],
        //U_counter.New_State[2:0],
        CountOut[4:0]);
end


endmodule
```

(c) 以 vivado 完成電路的 implement，並確認所占用的硬體資源是否合理。(Report

Utilization）（10%*2）

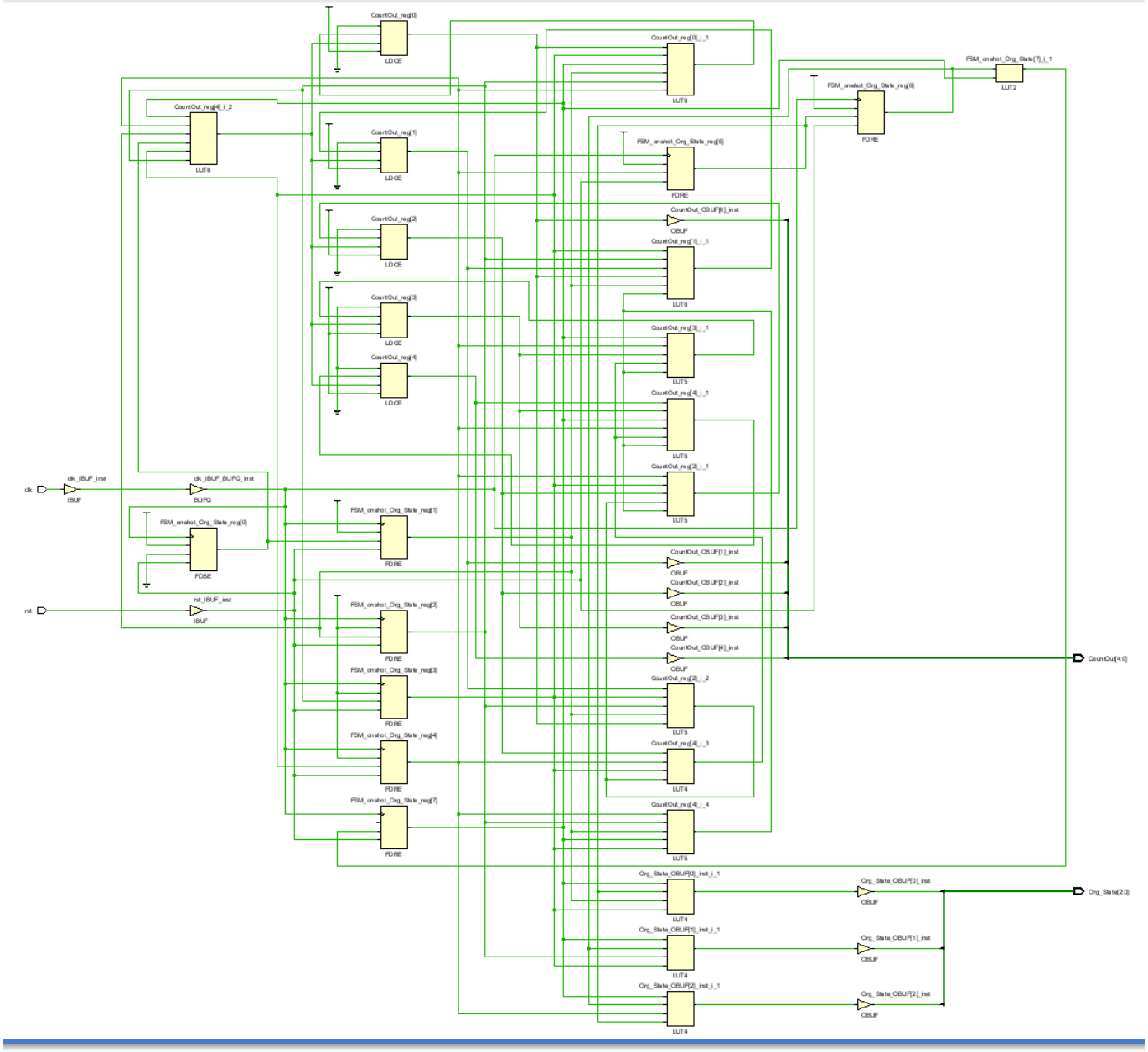


Utilization × Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing

Hierarchy

- Hierarchy
- Summary
- Slice Logic

Name	Slice LUTs (8000)	Slice Registers (16000)	Slice (3650)	LUT as Logic (8000)	Bonded IOB (150)	BUFGCTRL (32)
FSM_HW_2	11	13	5	11	10	1



Vivado implementation 後僅使用 11 個 LUT 與 13 個 Flip-Flop，整體資源使用量非常低。

電路未使用 BRAM 或 DSP 等大型硬體資源，顯示其設計符合簡單控制電路的特性。

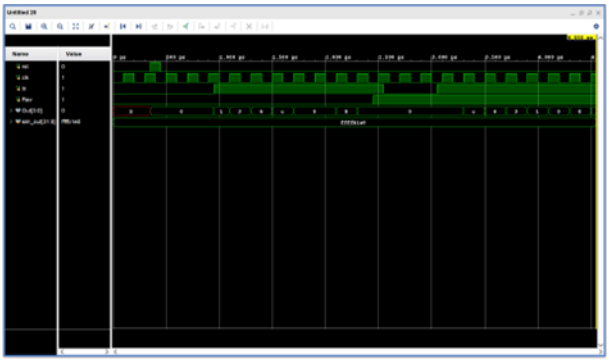
FPGA 上的資源使用屬於合理且有效率的範圍。

報告中完成以上所有項目可得 80 分。另有 20 分加分項目，可以包含記錄除錯過程，或進行額外 post implementation 模擬比較，也可將第一題的結果加上除頻電路，進行 FPGA 開發板燒錄，並以 LED 顯示輸出。

除錯紀錄：

第一題：

第一版結果



問題 1：顯示順序錯誤

原本輸出序列為：**1 → 3 → 6 → C → 0 → 0 → 8 → 0**

學號為：**0136C008**

修正 2：重新定義輸出對應

對應學號：

S0: Out = 4'b0000; // 0

S1: Out = 4'b0001; // 1

S2: Out = 4'b0011; // 3

S3: Out = 4'b0110; // 6

S4: Out = 4'b1100; // C

S5: Out = 4'b0000; // 0

S6: Out = 4'b0000; // 0

S7: Out = 4'b1000; // 8

問題 2：輸出會因 **Rev** 改變而立即跳動

原程式：**case ({Org_State, Rev})**

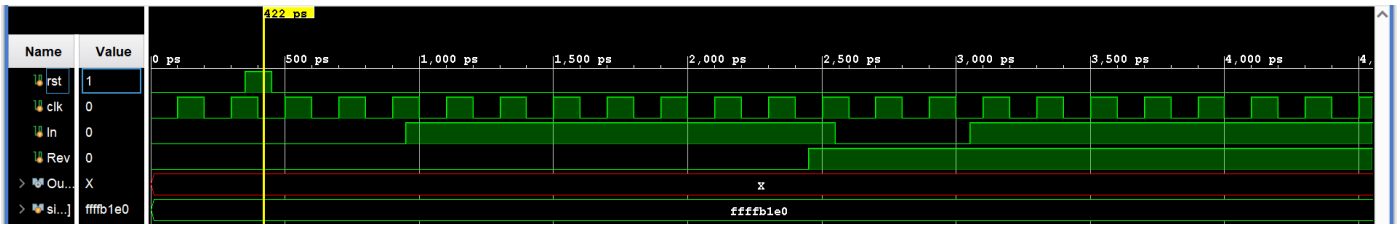
導致：

- 當 **Rev** 改變時
- 即使 **state** 沒變
- **Out** 也會立即改變

修正 2：輸出不再依賴 **Rev**

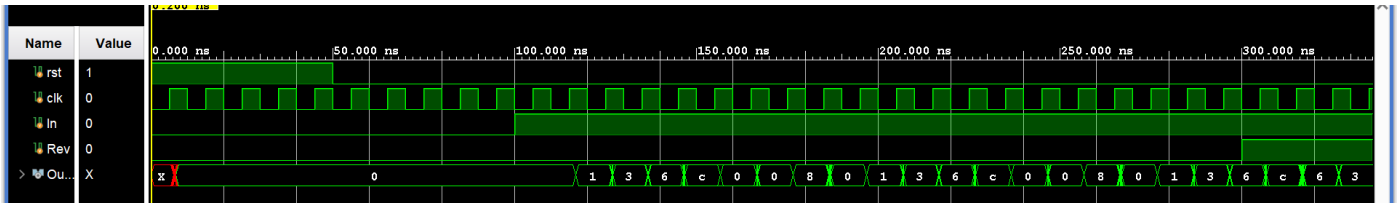
改為：**case (Org_State)**

問題 3：post implementation 模擬不如預期



原因：**testbench reset** 太短以及 **clk** 太短 **+ post-implementation timing simulation** 有實體延遲導致 **Out** 一直是 **X**

修正 3：延長 **reset** 以及 **clk**

post implementation 模擬：

第一題：